

Examen de Desarrollo de Sistemas Móviles

20 • 06 • 26

- Noa
- Quispe Mamani
- Código : 21200113

1) Desde mi punto de vista creo que se le considera así porque aquí es donde se centraliza el acceso a los datos, de esta manera se evita que la lógica de negocio acceda directamente a la base de datos. Se dice que actúa como fachada, justamente por lo antes mencionado, que no deja ver la complejidad de la base de datos y en cambio nos muestra una cara más simple.

El Repositorio coordina dos fuentes: Room y Retrofit. Primero consulta la base de datos local para mostrar datos rápido, y luego de realizar esta acción, se puede actualizar desde la API y se sincroniza la información. De esta manera, la UI solo observa los datos del repositorio sin saber si vienen de internet o de la misma base de datos.

2) • El viewmodelscope es una scope de corutinas que está ligado al ciclo de vida del viewmodel. Lo que nos dice que cualquier Corutina que se lance dentro de este scope será destruido cuando el Viewmodel es destruido. Esto es fundamental ya que así se evitan fugas de memoria y procesos que son innecesarios en segundo plano.

• Cuando se inicia una petición en este scope y el usuario cierra la pantalla antes de recibir la respuesta, la corutina se cancela automáticamente, por lo que la petición se detiene y tampoco consume recursos.

3) Flow: Este no realiza ninguna acción hasta que alguien se conecte (el collect). Cada recolector activa una nueva ejecución desde el principio. mientras que el state Flow: Este en contrario siempre está activo, y mantiene el último estado en memoria y los distribuye a todos los que se conectan.

• Se prefiere el StateFlow por que en una UI siempre se necesita saber el estado actual al instante y como es caliente o diferencial del Flow, no vuelve a realizar peticiones que a veces nos parezcan molestas ni a realizar procesos desde cero.

- 4) Este nos ayuda a controlar la creación de objetos globales en la Clase Application. Ya que esta clase arranca antes que cualquier otra cosa y vive durante todo el ciclo de vida de la app, se usa como un contenedor de dependencias. La palabra by lazy se usa para inicializar los objetos solo cuando se necesitan por primera vez. Esto ayuda a mejorar el rendimiento, ya que nos evita crear objetos pesados.

Completar:

- 1) El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow.
- 2) El operador dispatchers.io permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU como el procesamiento masivo de datos.
- 3) Para que un servicio de primer plano de ubicación sea válido en Android 10, debe declararse en el manifiesto con el tipo Location.
- 4) La función getInstance() de la Clase application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".
- 5) El componente que permite transformar una API basada en callbacks antiguos en una función suspendible se llama Suspend callable Coroutine.
- 6) El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml.
- 7) En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL (Hardware Abstraction Layer).
- 8) El operador de flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es Combine.

III: Desarrollo de Código y lógica Personalizada.

1) Interface Usuario Dao {

}

Class Usuario Repositorio (private val usuarioDao: usuario Dao) {

fun obtenerUsuarios() = usuario Dao . obtenerUsuarios()

fun insertar Usuario (usuario: usuario) = usuario Dao . insertar (usuario)

Instancia en Application usando by lazy.

Class MyApp: Application() {

val usuarioRepository by lazy {
 Usuario Repositorio (usuario Dao)

}

val usuarioDao by lazy {

App Database . getInstance (this) . usuario Dao ()

}

}

2) a)

data class Usuario (

val nombre: String?,
val apellidos: String?,
val edad: Int?

b)

val usuario = Usuario (

nombre = "Poa",

apellido = "Guilherme Yamani"

edad = 25)

c)

```
val puntajeIdentidad =  
(usuario.nombre?.length?:0) +  
(usuario.apellidos?.length?:0)
```

d)

```
val nombreCompleto = "${usuario.nombre?:""}${usuario.  
    apellidos?:""}"  
  
println("Usuario: $nombreCompleto, tu puntaje de Identidad  
    es: $puntajeIdentidad")
```

e)

Usuario: Noe Quispe Mamani, tu puntaje de Identidad es: 15.

IV: Architecture de AOSP:

1)

1: Applications
2: Application Framework
3: Android Runtime (ART)
4: Native Libraries
5: Linux Kernel

2)

1) Applications.

Es la capa superior donde están las aplicaciones del usuario (WhatsApp, Chrome, etc). Usa las APIs del framework para funcionar.

2) Application Framework.

Proporciona los servicios principales para las apps, como:

- Activity Manager.
- Window Manager.
- Location Manager.
- Content Providers.
- Notification Manager.

Es la capa que permite que las apps no accedan directamente al hardware.

3) Android Runtime (ART)

Es el entorno donde se ejecutan las aplicaciones Android.

- Componentes: Dex bytecode execution
- Garbage collector (GC)
- Core Libraries (Java/Kotlin APIs).

• ART se encarga de convertir el código de las apps en ejecución eficiente.

4) Native Libraries.

Contiene librerías en C/C++ usadas por el sistema.

- SQLite (base de datos)
- OpenGL ES (Gráficos)
- WebKit / Chromium.
- Media framework.

5) Linux Kernel.

Es la base del Sistema Operativo.

- Gestión de Memoria
- Proceso de hilos
- Drivers de hardware
- Seguridad
- Energía y red

"Antes que nada, la preparación es la llave del éxito" - Alexander Graham Bell

Universal